

dev.gitlab.org - Maintenance tasks

The Release&Deploy team is responsible for ensuring that the GitLab instance on this server remains operational.

Requirements

- Access to the node
- Depending on whether the task requires permanent changes to `/etc/gitlab/gitlab.rb`, access to the Chef repo. If you do not have access to this repository, make sure you create an issue in Infrastructure issue tracker and label it `access request`.

Manually upgrading/downgrading packages

Downgrading packages

In case of an issue with the latest deploy, we might need to revert the installation to a previous nightly version and lock the deployment until the fixes are ready. This is done to ensure stability of dev.gitlab.org for others using the instance.

1. Create an issue in the release tasks issue tracker describing the downgrade and including links to related issues. Assign the issue to yourself.
2. Announce the downgrade in the `#announcements` Slack channel before proceeding:

I will be manually downgrading package on dev.gitlab.org to `<version>` as latest nightly

3. Stop Sidekiq and Puma to prevent data from being altered during the downgrade:

```
sudo gitlab-ctl stop sidekiq
sudo gitlab-ctl stop puma
```

4. Find the previous working version. Each time a package is installed on the instance, a message is sent to the `#announcements` Slack channel. For example:

Package upgrade finished on host dev-1-01-sv-dev-1, current installed version is 18.8.

5. Downgrade to the previous version:

```
sudo apt-get install gitlab-ce=<version>
```

For example, to downgrade to version `10.4.0+rnightly.75436.44501791-0`, run:

```
sudo apt-get install gitlab-ce=10.4.0+rnightly.75436.44501791-0
```

This will automatically run reconfigure and apply the necessary changes.

6. After reconfiguration completes, restart all services:

```
sudo gitlab-ctl restart
```

7. Verify all services are running:

```
sudo gitlab-ctl status
```

8. Verify the correct version is deployed by visiting <https://dev.gitlab.org/help>.

9. Create a package hold to prevent automatic upgrades:

```
sudo apt-mark hold gitlab-ce
```

Verify the hold is in place:

```
sudo apt-mark showhold
```

10. Post a message in the `#announcements` channel confirming the downgrade is complete:

Downgrade completed. The package has also been put on hold to prevent automatic upgrades

11. Close the issue and add the following labels:

- `release-blocker`
- `Deploys-blocked-gstg`
- `Deploys-blocked-gprd`

Upgrading packages

Once the issue has been resolved, unhold the package and upgrade to the latest version.

1. Announce the upgrade in the `#announcements` channel:

I will be removing the package hold and manually upgrading package on dev.gitlab.org to

2. Unhold the package:

```
sudo apt-mark unhold gitlab-ce
```

3. Continue with upgrading:

```
sudo apt-get update
```

```
sudo apt-get install gitlab-ce
```

4. Verify all services are running:

```
sudo gitlab-ctl status
```

5. Verify the latest version is installed by visiting <https://dev.gitlab.org/help>.

6. Finally, leave a note in the `#announcements` channel:

Upgrade completed. dev.gitlab.org now runs `<version>`. `<link to issue>`

Changing GitLab configuration

If, for some reason, you need to apply a change to `/etc/gitlab/gitlab.rb`, this change needs to be introduced in the dev-gitlab-org role.

Exceptional process

[!Warning] The following steps are exceptional and should only be used as a last resort by team members who don't have access to make MRs to the chef-repo codebase.

If you do not have access to this repository, but you need to do a hot-patch or configuration testing, the following steps can be performed:

1. Stop chef-client on this node:

```
sudo service chef-client stop
```

2. Make the necessary changes to restore the instance. If changes to the `gitlab.rb` file are required, edit it manually and run `reconfigure`.
3. Reach out to Production team to get help on getting your `gitlab.rb` configuration change committed to the Chef server.
4. After the changes are applied, start the chef-client on the node:

```
sudo service chef-client start
```

5. Make sure that any change you did is noted in an issue! It is your responsibility to revert the change on this node once the fix is in place in the package!

Failed update due to malformed configuration

When a previous update failed, it may leave behind an incomplected configuration. Then, when the next update (`apt install gitlab-ce`) happens, it fails with the error:

Malformed configuration JSON file found at `/opt/gitlab/embedded/nodes/dev-1-01-sv-dev-1.c.g`

Solution

Make a backup of the malformed configuration:

```
sudo cp /opt/gitlab/embedded/nodes/dev-1-01-sv-dev-1.c.gitlab-dev-1.internal.json /tmp/
```

Remove the malformed configuration:

```
sudo rm /opt/gitlab/embedded/nodes/dev-1-01-sv-dev-1.c.gitlab-dev-1.internal.json
```

Rerun the update again:

```
sudo apt update
```

```
sudo apt install gitlab-ce
```

Once the update succeeds, validate if the instance is functional by accessing <https://dev.gitlab.org/help>. You may need to wait a bit when the GitLab process is restarting.

Remove the backup configuration file:

```
rm /tmp/dev-1-01-sv-dev-1.c.gitlab-dev-1.internal.json
```